ADAPT: Adaptive Semantic Joins with Verification Feedback and Anytime-Valid Quality Guarantees

Zhaokun Zhang[1], Zhaoyan Sun[1], Yong Zhang[1], Chunxiao Xing[1], Yaoqiang Xu[2], Chen Qian[2]
[1]Department of Computer Science, Tsinghua University; [2]East Branch of State Grid Corporation of China
{zhangzk24,szy22}@mails.tsinghua.edu.cn,{zhangyong05,xingcx}@tsinghua.edu.cn
xuyq99@139.com,arielqc@163.com

***Abstract*—Semantic joins identify pairs of records from two textual collections that satisfy natural-language predicates, but their large join spaces and low selectivity make LLM-based verification prohibitively expensive. Existing approaches reduce this cost following a fixed filter-then-verify pipeline: a filter generates candidate pairs, and LLMs verify them. However, the abundant LLM feedback obtained during the verification stage remains underutilized for improving the candidate generation capability of the filter. In this paper, we present ADAPT, an adaptive feedback-aware semantic join framework that progressively discovers matching pairs over multiple rounds of iteration and uses verification feedback to guide candidate generation in subsequent rounds. Specifically, ADAPT extracts multi-channel features and embeds them into a shared embedding space for candidate generation. During iteration, ADAPT uses a multi-armed bandit policy to select promising candidates for LLM verification, and further trains lightweight residual MLPs from the verification results to refine the embeddings and improve subsequent candidate generation. To determine when to stop, ADAPT first provides a *certified mode* that conducts stratified auditing with additional LLM cost to offer anytime-valid quality guarantees for user-specified targets. We also derive a lower bound on the LLM cost required for this guarantee and provide a *heuristic mode* for stopping to avoid this cost. Experiments on six real world datasets show that ADAPT achieves higher ground-truth F1 than state-of-the-art baselines. *Certified mode* reduces LLM monetary cost by 2.1–$8.8\times$, while *heuristic mode* reduces cost by 3.7–$12.7\times$.**

***Index Terms*—Adaptive query processing, anytime-valid inference, large language models, quality guarantees, semantic joins.**

I. INTRODUCTION

The rapid progress of Large Language Models (LLMs) has enabled semantic-operator systems for analyzing large collections of unstructured documents [1]–[10]. These systems extend traditional relational operators with natural-language predicates evaluated by LLMs. A core operation is the *semantic join*: given two collections of textual records, L and R, and a natural-language predicate p, it returns all record pairs $(l_i \in L, r_j \in R)$ that satisfy p based on an LLM. Similar to conventional joins in OLAP workloads, semantic joins can dominate query execution cost. A straightforward nested-loop implementation evaluates the predicate for every pair, requiring $|L| \times |R|$ LLM calls. This can be prohibitively expensive in both monetary cost and latency, making efficient semantic join processing essential for practical deployment.

Existing systems [1], [4], [11] typically use a two-stage *filter-then-verify* workflow (Figure 1(a)). First, the *filtering stage* uses sampled pairs to construct and calibrate a filter

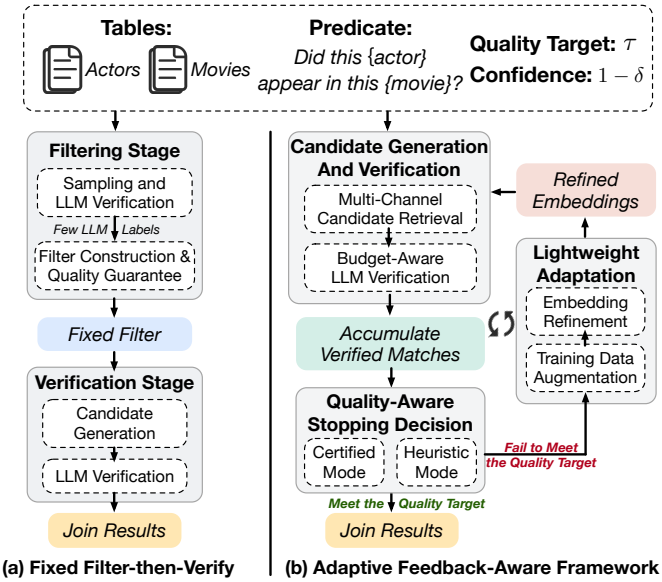


Fig. 1. Semantic-Join Framework Comparison. (a) Fixed Filter-then-Verify. (b) Adaptive Feedback-aware Framework.

that generates candidates and provides a quality guarantee using these labels. Second, the *verification stage* uses an LLM to evaluate each candidate and adds confirmed matches to the join result. However, the filter remains fixed throughout verification: newly verified labels are not fed back to refine candidate generation. This limitation is particularly severe for low-selectivity semantic joins, where only a small fraction of record pairs are matches. Initial samples may contain few matches, providing insufficient supervision to build a filter with guarantee that generalizes across the join space. Although verification produces additional labels, the fixed filter cannot exploit this feedback to improve subsequent candidate generation. Specifically, LOTUS's embedding-based cascades [1] and DocETL's embedding-based blocking [4] rely primarily on similarity scores, which may not accurately capture the semantics of the join predicate and can therefore struggle to distinguish matches from others. FDJ extracts predicate-relevant features using LLMs and combines feature comparisons into logical filters [11], but extensive feature extraction and filter calibration can incur substantial LLM costs.

Motivation. Our key observation is that the LLM judgments collected during verification provide valuable supervision for refining the filters and identifying promising candidate pairs. Instead of treating verification as the final step, we can use its

feedback to iteratively refine the filter step and discover additional matches. This motivates an *adaptive feedback-aware semantic join framework* that continuously reuses verification feedback to find more matches with fewer LLM calls.

Challenges. Realizing this framework requires addressing three challenges:

C1: Feedback-Guided Candidate Selection and Verification. Semantic joins often have a large search space with sparse matches, making candidate selection critical for controlling LLM costs. However, verification serves two purposes: discovering matches and collecting supervision for improving future candidate retrieval. The pair most likely to be a match is not necessarily the pair that provides the most useful feedback for refining the filter step. The challenge is therefore to select candidates that balance the immediate value of discovering matches with the longer-term value of improving future candidate discovery.

C2: Lightweight and Effective Filter Refinement. Verification feedback is useful only if it can be translated into better candidate generation, but updating the filter step can itself be expensive. Fine-tuning an embedding model requires additional training and re-encoding, while approaches such as FDJ [11] incur substantial LLM costs for feature extraction and filter construction. The challenge is therefore to effectively incorporate accumulated verification labels into filtering without introducing significant update overhead.

C3: Stopping with a Valid Quality Guarantee. The system must determine whether the discovered matches are sufficient to meet a user-specified quality target, even though undiscovered matches remain in the unverified search space. This is challenging because candidate selection is adaptive: verification results influence subsequent filtering, which in turn changes which pairs are verified next and the composition of the remaining search space. Moreover, the system may repeatedly evaluate whether to stop. The challenge is therefore to develop a quality guarantee that remains statistically valid under adaptive process and holds at the time the system stops.

To address these challenges, we propose ADAPT, an adaptive feedback-aware semantic join framework with quality guarantees (Figure 1(b)).

For **C1**, ADAPT integrates candidate generation, LLM verification, and feedback-driven adaptation into a progressive iteration. In each round, it generates candidates using index retrieval based on multi-channel features that capture different matching patterns. An adaptive multi-armed bandit policy uses recent verification outcomes to allocate the shared verification budget across channels, favoring channels with higher estimated match rates while continuing to explore less-certain ones. This strategy balances exploration and exploitation to discover more matches under a limited verification budget.

For **C2**, ADAPT shares verification feedback across channels, allowing each channel to learn from matches and non-matches discovered by the others. A lightweight residual MLP refines only the driving-side embeddings, while keeping the pretrained embedding model, driven-side embeddings, and retrieval index fixed. This design incorporates new labels into candidate generation with low computational overhead and without additional LLM calls.

For **C3**, ADAPT combines confirmed matches with stratified samples from the unverified pairs to construct an upper confidence bound on the total number of matches. This bound yields an anytime-valid lower bound on recall, enabling certified stopping even as retrieval adapts over time. We also derive a lower bound on the verification cost required for certified completion. For applications that do not require a formal guarantee, ADAPT provides a *heuristic mode* that estimates recall from existing verification feedback, further reducing LLM costs.

Contributions. We summarize our contributions as follows:

- **Adaptive Feedback-Aware Semantic Join Framework.** We propose a progressive execution framework ADAPT that continuously accumulates verified matches and uses verification feedback to improve subsequent candidate generation and allocate the verification budget. ADAPT integrates this adaptive iteration with quality-aware stopping criteria (Section III).
- **Anytime-Valid Quality Certification.** We develop a *certified mode* that provides a recall guarantee over the full join space and remains valid across iterations despite adaptive retrieval updates. We also establish a lower bound on the verification cost of certified completion and offer a lower-cost *heuristic mode* without a formal guarantee (Section IV).
- **Lightweight Multi-Channel Feature Refinement.** We design complementary multi-channel retrieval which shares verified pairs as supervision. Lightweight residual MLP refinement updates only the driving-side embeddings, preserving the driven-side embeddings and retrieval index to avoid re-embedding and index rebuilding (Section V).
- **Empirical Evaluation.** We evaluate ADAPT against state-of-the-art baselines on six semantic join datasets. Both execution modes achieve higher F1 scores than all baselines. Against the per-dataset least-cost baseline, certified and heuristic modes reduce LLM monetary costs by 2.1–8.8× and 3.7–12.7×, respectively (Section VI).

II. PRELIMINARIES

Semantic Join. A semantic join takes two textual tables $L = \{l_1, \ldots, l_m\}$ and $R = \{r_1, \ldots, r_n\}$, a natural-language predicate p, and an LLM M. The predicate p specifies the semantic relationship that should hold between a pair of records (l_i, r_j). Given a pair, the LLM returns $M_p(l_i, r_j) \in 0, 1$, indicating whether the pair satisfies p. The exact semantic join result is therefore

$$J = \{(l_i, r_j) \in L \times R \mid M_p(l_i, r_j) = 1\}. \tag{1}$$

For example, consider an actor collection L and a movie collection R, the predicate "Did this actor appear in this movie?" joins an actor record with a movie record if the actor appeared in that movie.

An exhaustive nested-loop evaluation checks all $|L| \times |R|$ record pairs, requiring one costly LLM call per pair. This

motivates approximate semantic join algorithms that reduce LLM inference cost while maintaining high result quality.

Approximate Semantic Join with Quality Guarantee. Given an approximate semantic join algorithm $\mathcal{A}$ that returns a result $\widehat{J} \subseteq L \times R$, we measure its result quality using the F1 score relative to LLM labels:

$$Q(\mathcal{A}) = \frac{2|\widehat{J} \cap J|}{|\widehat{J}| + |J|}. \tag{2}$$

We measure the LLM inference cost of $\mathcal{A}$ by the total number of tokens consumed across all calls to M:

$$C(\mathcal{A}; L, R, p, M). \tag{3}$$

Definition 1 (Quality-Guaranteed Approximate Semantic Join)**.** Given two textual collections L and R, a semantic join predicate p, an LLM M, a target quality level τ, and a failure probability δ, the quality-guaranteed approximate semantic join problem is to design an algorithm $\mathcal{A}$ that minimizes the LLM inference cost while ensuring that the returned result achieves quality at least τ with probability at least $1 - \delta$:

$$\begin{aligned} &\underset{\mathcal{A}}{\text{minimize}} \quad C(\mathcal{A}; L, R, p, M) \\ &\text{subject to} \quad \Pr[Q(\mathcal{A}) \geq \tau] \geq 1 - \delta. \end{aligned} \tag{4}$$

III. System Overview

We propose ADAPT, an adaptive feedback-aware semantic join framework that progressively constructs the join result $\widehat{J}$ while controlling result quality. As shown in Figure 2, ADAPT iterates through four steps:

Step 1: Multi-Channel Feature Extraction on the Driving Side and Index Construction on the Driven Side. ADAPT distinguishes the driving table L and the driven table R from the two input tables by leveraging their semantic contents and the predicate. It first samples records from both tables, assembles them with the predicate into a prompt, and invokes one LLM call to produce table descriptions. Using these descriptions, the predicate, and the sampled records, ADAPT selects L and R using LLM according to three rules: (1) predicting an R record from an L record via the predicate should be simpler; (2) L records should have longer text and richer semantic information; (3) If the above two rules are inconclusive, the table with fewer records is chosen as L.

On the driving side, ADAPT extracts features from each record l_i through four channels. These channels capture matching signals at different granularities and provide a predicate-aware representation of potential matches: (1) *Word*: keywords selected through two-stage filtering. First, TF-IDF removes overly rare and frequent terms. Second, only words whose maximum embedding cosine similarity to any R record exceeds a predefined threshold are retained; (2) *Sentence*: text chunks that capture sentence-level matching signals; (3) *Document*: the complete record, which captures document-level matching signals; and (4) *LLM extraction*: phrases describing potential matches in R, generated by prompting an LLM with l_i, the join predicate, and two table descriptions. Each feature is encoded separately using a pretrained embedding model.

On the driven side, ADAPT encodes each complete record r_j into a single embedding and builds a shared index over these embeddings. This asymmetric design avoids searching up to 4×4 channel combinations, reducing retrieval overhead and preventing duplicate candidate-pair retrievals.

Step 2: Candidate Generation and Verification. In each round, ADAPT constructs a candidate queue for each feature channel and dynamically allocates a total budget of T LLM calls across these queues to verify previously unverified pairs.

(2.1) Candidate Queue Generation. ADAPT constructs a candidate queue for each channel in two stages: *retrieval* and *ranking*. In the first round, retrieval uses the initial driving-side embeddings; in subsequent rounds, it uses the embeddings refined in Step 4. In all rounds, retrieval is conducted over the shared driven-side index.

In the *retrieval stage*, each driving-side record may have one or multiple feature embeddings depending on the channel. For each channel, ADAPT uses each feature embedding to independently retrieve the top-K most similar unverified driven-side records. Driving-side record associated with this feature embedding and the retrieved driven-side record form a candidate pair, whose cosine similarity is recorded. All pairs constitute the candidate pool for this channel.

In the *ranking stage*, ADAPT deduplicates pairs in each channel's candidate pool, keeping the maximum cosine similarity among duplicates as the score. It then sorts the pairs of each channel independently by cosine similarity and retains the top-T pairs to construct the candidate queue for the channel.

(2.2) LLM Verification with Dynamic Budget Allocation. Given the channel-specific queues, ADAPT selects candidates for verification under the shared budget. Because similarity scores across channels are not directly comparable and channels vary in their ability to identify matches, ADAPT uses real-time LLM labels to guide budget allocation.

Specifically, ADAPT formulates channel selection as a multi-armed bandit problem. Each channel queue is an arm, and verifying a candidate yields a binary reward: 1 for a match and 0 for a non-match. As candidates are processed in descending order of similarity, a queue's match rate may decline. To adapt to such variations in match rate, ADAPT adopts Sliding-Window UCB (SW-UCB) [12]. For each channel c, ADAPT maintains a sliding window of the most recent W verified pairs to track its recent match rate. Let $n_c(t)$ denote the number of verified pairs from channel c within the window, and let $\widehat{p}_c(t)$ denote their match rate. After t pair verifications in the current round, ADAPT computes the channel's score as $\widehat{p}_c(t) + \sqrt{\frac{\alpha \ln(\min\{t,W\})}{n_c(t)}}$, where α controls the exploration strength. The first term favors channels that have recently yielded more matches, while the second encourages exploration of channels with fewer recent verifications. At each step, ADAPT selects the channel with the highest score and verifies the first unverified pair in its candidate queue. This strategy adaptively balances exploitation and exploration based on recent LLM feedback.

Confirmed matches are added to the accumulated join result.

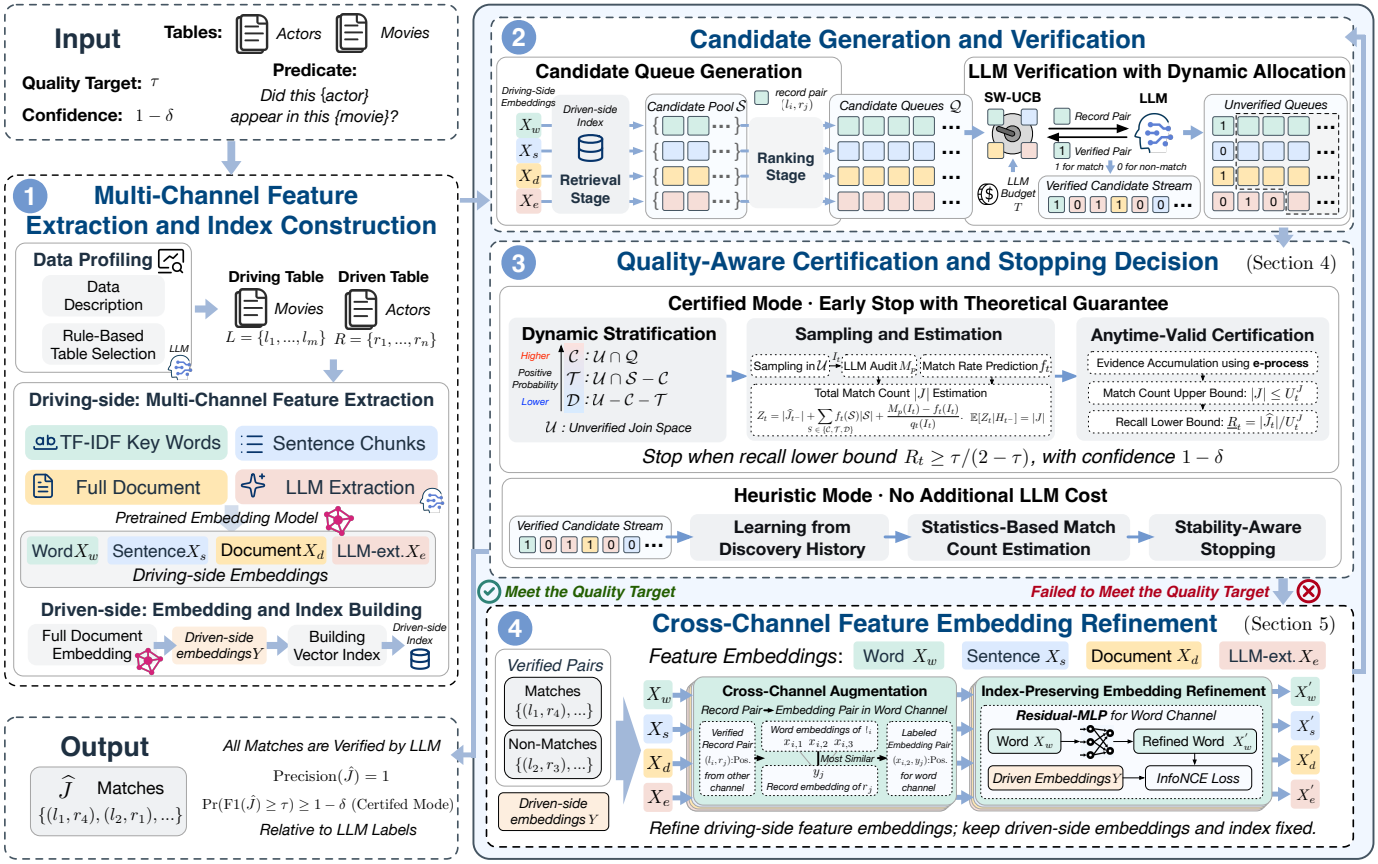


Fig. 2. System framework of ADAPT.

All verified pairs and their labels are retained in a cumulative *verified candidate stream*, which supports the stopping decision in Step 3 and embedding refinement in Step 4.

Step 3: Quality-Aware Certification and Stopping Decision. After each verification round, ADAPT checks whether the accumulated join result $\widehat{J}$ meets the target quality. Since all matches in $\widehat{J}$ are verified by the LLM, the precision of $\widehat{J}$ is 1, and the quality target reduces to a recall requirement. It offers two stopping modes with different quality–cost trade-offs: *Certified mode* provides anytime-valid guarantees with additional LLM cost. *Heuristic mode* reduces overhead via quality estimation on existing LLM results, without formal guarantees. If the selected stopping criterion is met, ADAPT returns $\widehat{J}$. Otherwise, it proceeds to Step 4 to refine the driving-side embeddings using the accumulated verified pairs.

Certified Mode. Certifying recall requires an upper bound on the total number of matching pairs $|J|$, including unverified matches. To construct this bound, ADAPT partitions the unverified join space into three regions, as described in Section IV-B, and samples pairs for LLM verification. From sampled labels and sampling probabilities, ADAPT use an e-process [13] to construct an anytime-valid upper bound on $|J|$ with probability at least $1-\delta$. Dividing the number of confirmed matches $|\widehat{J}|$ by this upper bound yields a lower confidence bound on recall. ADAPT stops when this lower bound reaches the target.

Heuristic Mode. Rather than conducting additional LLM cost, ADAPT uses the *verified candidate stream* to model match probabilities and estimate the total match counts in the full join space. It divides $|\widehat{J}|$ by a conservative estimate of the total number of matches to obtain a heuristic recall estimate. Unlike the certified lower bound, this estimate provides no formal guarantee. ADAPT then applies the stopping rules described in Section IV-C to determine whether the estimated recall provides sufficient evidence to stop at the target.

Step 4: Cross-Channel Feature Embedding Refinement. ADAPT uses the accumulated verification feedback to improve candidate retrieval by refining the driving-side feature embeddings, keeping the pretrained embedding model, driven-side embeddings and index fixed. There are two substeps.

(4.1) Cross-Channel Augmentation. To share supervision across channels, ADAPT maps verified record pairs to feature-to-record training examples for each channel. For example, suppose the document channel retrieves (l_i, r_j), verified as a match. The driven record r_j has a single embedding y_j, whereas the driving record l_i may have multiple word embeddings. To construct a training example for the word channel, ADAPT selects the word embedding most similar to y_j and pairs it with y_j, assigning label 1. Verified nonmatches are mapped in the same way and assigned label 0. Applying this procedure to the accumulated verified pairs allows each channel to learn from candidates retrieved by other channels.

(4.2) Index-Preserving Embedding Refinement. Each channel trains a small residual MLP to transform its driving-side feature embeddings as described in Section V. In each round, the MLP is initialized to the identity mapping and trained on the accumulated supervision, including the cross-

channel examples, using an InfoNCE loss [14].

Then, ADAPT returns to Step 2 and uses the refined driving-side embeddings to retrieve additional candidate pairs.

IV. Quality-Aware Certification and Stopping Decision

During progressive execution, ADAPT determines whether the current join result $\widehat{J}$ has reached the target quality and whether the iteration can terminate. To formalize this decision, we first define the anytime-valid quality guarantee with respect to LLM labels, and derive a lower bound on the end-to-end LLM verification cost of certified approximate semantic joins (Section IV-A). We then introduce two stopping modes that trade off quality guarantees against LLM cost: **Certified mode** (Section IV-B) provides an anytime-valid quality guarantee through randomized *auditing* incurring additional LLM cost. **Heuristic mode** (Section IV-C) avoids this additional cost by estimating quality from previously verified pairs with no formal quality guarantee.

A. Anytime-Valid LLM Verification Stopping

During progressive execution, ADAPT repeatedly checks whether the current join result meets the quality target τ. We establish an *anytime-valid* guarantee relative to *LLM labels*: the probability that the system ever stops with quality below τ is at most δ, even when the stopping decision depends on feedback from previous verifications. Achieving this guarantee may require randomized *auditing*, which samples unverified pairs with nonzero probabilities to gather evidence about matches that have not yet been discovered. In this section, we formalize the quality guarantee and derive a lower bound on the end-to-end LLM verification cost required to certify the quality of an approximate semantic join.

From F1 Target to Recall Target. Since ADAPT returns only pairs that the LLM has verified as matches, we have $\widehat{J} \subseteq J$, ensuring no false positives relative to the LLM labels. Assuming $|J| > 0$, we define recall as $\mathcal{R} = |\widehat{J}|/|J|$. By Equation 2, the F1 score is therefore determined solely by recall:

$$Q = \frac{2|\widehat{J} \cap J|}{|\widehat{J}| + |J|} = \frac{2|\widehat{J}|}{|\widehat{J}| + |J|} = \frac{2\mathcal{R}}{1+\mathcal{R}}. \tag{5}$$

Thus, meeting the F1 target $Q \geq \tau$ is equivalent to meeting the recall target

$$Q \geq \tau \iff \mathcal{R} \geq \tau_{\mathcal{R}}, \quad \text{where } \tau_{\mathcal{R}} = \frac{\tau}{2-\tau}. \tag{6}$$

To determine whether the recall target has been met, ADAPT must account for undiscovered matches, since the total number of matches $|J|$ is unknown. To gather this evidence, ADAPT conducts randomized auditing via an ordered sequence of sampling events $t = 1, 2, \ldots$. We denote the t-th sampling event as audit t, in which ADAPT samples unverified candidate pairs and obtains their LLM labels. After each audit, ADAPT adds any newly verified matches to the join result, updates the evidence used for certification, and checks whether the recall target has been met. The quality guarantee must hold across all such checks, as formalized below.

Definition 2 (Anytime-Valid Stopping with Quality Guarantee)**.** Let $\widehat{J}_t$ denote the updated join result after audit t, and let $\textsc{Stop}_t$ denote the event that ADAPT certifies $\widehat{J}_t$ and terminates after this audit. A stopping rule is $(\tau_{\mathcal{R}}, \delta)$-anytime-valid if

$$\Pr\left(\exists t: \ \textsc{Stop}_t \ \wedge \ \frac{|\widehat{J}_t|}{|J|} < \tau_{\mathcal{R}}\right) \leq \delta. \tag{7}$$

The probability is taken over the execution's random choices.

Definition 2 guarantees that the probability of stopping and declaring the recall target met when the actual recall is below the target is at most δ, satisfying the quality requirement in Equation 4. Here, *anytime-valid* means that the system can repeatedly evaluate the accumulated evidence and decide whether to stop or continue verification at any time, without compromising the guarantee. In contrast, conventional fixed-sample statistical guarantees, such as those based on p-values or confidence intervals, control errors at a prespecified sample size but do not generally remain valid under repeated testing or adaptive stopping. Section IV-B presents a certification method that supports this anytime-valid stopping rule.

End-to-End LLM Verification Cost Lower Bound. We provide the LLM verification lower bound in Theorem 1. Let S denote the total cost of LLM verifications and $N = |L||R|$ denote the total number of pairs, $\kappa = |J|/N$ be the join selectivity. We focus on sparse semantic joins, where $\kappa < \tau_{\mathcal{R}}$.

Theorem 1 (End-to-End Verification Cost Lower Bound)**.** *Let $\mathbb{E}[S]$ be the expected total number of pair-level LLM verifications before termination. For any algorithm meeting the quality requirement in Definition 2, the lower-bounded cost is:*

$$\mathbb{E}[S] \geq \underbrace{(1-\delta)\lceil \tau_{\mathcal{R}} \kappa N \rceil}_{\textit{discovery cost}} + \underbrace{\left(1-\delta^{1/H}\right)\left((1-\kappa)N - H + 1\right)}_{\textit{audit cost}}, \tag{8}$$

where

$$H = \left\lfloor \frac{1-\tau_{\mathcal{R}}}{\tau_{\mathcal{R}}} \kappa N \right\rfloor + 1.$$

The proof is provided in Appendix A. The lower bound captures two necessary sources of verification cost. First, the *discovery cost*: any successful execution must verify at least $\lceil \tau_{\mathcal{R}} \kappa N \rceil$ matching pairs. In ADAPT, these matches are primarily discovered through multi-channel retrieval and verification. Second, the *audit cost*: the system must sample additional pairs from the remaining search space and obtain sufficient evidence to bound the number of undiscovered matches, thereby certifying that the returned result satisfies the quality requirement in Definition 1.

For large-scale joins involving two input tables with hundreds or more records each, the *audit cost* term in Equation 8 can be simplified. For fixed quality parameters $(\tau_{\mathcal{R}}, \delta)$, H scales with total matches count κN, while $\log(1/\delta)$ remains constant and $N = |L||R|$ grows as the product of the two table sizes. Consequently, when κ is sufficiently small, H

becomes much larger than $\log(1/\delta)$ while remaining much smaller than N. Under these conditions, the audit cost lower bound simplifies to the expression given in Theorem 2.

Theorem 2 (Audit Cost for Large-Scale Joins)**.** *For large-scale semantic joins in the regime where H is much larger than $\log(1/\delta)$ and much smaller than N, the audit cost part in Equation 8 simplifies to:*

$$\Omega\left(\frac{1-\kappa}{\kappa}\frac{\tau_{\mathcal{R}}}{1-\tau_{\mathcal{R}}}\log\frac{1}{\delta}\right). \tag{9}$$

The proof is provided in Appendix B. Importantly, this lower bound does not explicitly depend on the join-space size N. Instead, for fixed quality parameters $(\tau_{\mathcal{R}}, \delta)$, it grows as $\Omega(1/\kappa)$ as the join selectivity κ decreases. This result highlights a fundamental challenge: even with optimal match discovery, certifying the quality of a large-scale semantic join with low selectivity can require substantial additional LLM verification cost.

B. Certified Mode: Early Stop with Theoretical Guarantee

To satisfy Definition 2 under adaptive execution, ADAPT must determine whether the current join result meets the recall target $\tau_{\mathcal{R}}$, while ensuring that the probability of stopping prematurely is at most δ. The key challenge is estimating the unknown total number of matches $|J|$ with a statistical guarantee that remains valid across repeated certification checks. Our certification procedure consists of three steps: (i) Total Match Count Estimation. (ii) Anytime-Valid Upper Bound on $|J|$. (iii) Certified Stopping. We first describe these three steps and establish their theoretical guarantee, then introduce techniques to further reduce LLM auditing cost.

Total Match Count Estimation. To construct the upper bound U_t^J used for recall certification, we first develop an estimator of the total match count $|J|$. Let $\mathcal{H}_{t^-}$, $\widehat{J}_{t^-}$, and $\mathcal{U}_{t^-}$ denote the execution history, the set of verified matches, and the remaining unverified join space before audit t, respectively. For each pair $x \in \mathcal{U}_{t^-}$, let $f_t(x) \in [0,1]$ denote its predicted match probability and $q_t(x) > 0$ its sampling probability, where $\sum_{x\in\mathcal{U}_{t^-}} q_t(x) = 1$. Both $f_t(x)$ and $q_t(x)$ are determined by the execution history $\mathcal{H}_{t^-}$. During audit t, we sample a pair I_t from $\mathcal{U}_{t^-}$ according to q_t and obtain its LLM label $M_p(I_t)$. The key idea is to estimate $|J|$ by combining the known number of verified matches $|\widehat{J}_{t^-}|$ with the predicted number of remaining matches $\sum_{x\in\mathcal{U}_{t^-}} f_t(x)$, and then correcting this prediction using the observed LLM label $M_p(I_t)$. This yields the following estimator of the total match count:

$$Z_t = |\widehat{J}_{t^-}| + \sum_{x\in\mathcal{U}_{t^-}} f_t(x) + \frac{M_p(I_t) - f_t(I_t)}{q_t(I_t)}. \tag{10}$$

The last term corrects the prediction error through inverse-probability weighting, making Z_t conditionally unbiased for $|J|$ [15], [16].

Theorem 3 (Conditional Unbiasedness and Boundedness of Z_t)**.** *For any audit t, the estimator Z_t in Equation 10 is conditionally unbiased for $|J|$ and admits a finite upper bound B_t computable from $\mathcal{H}_{t^-}$ before audit t:*

$$\mathbb{E}[Z_t \mid \mathcal{H}_{t^-}] = |J|, \qquad Z_t \le B_t. \tag{11}$$

Here,

$$B_t = |\widehat{J}_{t^-}| + \sum_{x\in\mathcal{U}_{t^-}} f_t(x) + \max_{x\in\mathcal{U}_{t^-}} \frac{1-f_t(x)}{q_t(x)}. \tag{12}$$

These properties continue to hold under adaptive retrieval and refinement, allowing audit evidence to accumulate across rounds while estimating the same fixed total match count $|J|$. The proof is provided in Appendix C.

The sequence of estimators $(Z_t)_{t\ge1}$ forms a stochastic process, where each estimate has a conditional expectation equal to the fixed total match count $|J|$. However, conditional unbiasedness alone does not guarantee that *the probability of an incorrect stopping decision is at most δ*, particularly under repeated certification checks. To address this challenge, we use the estimates Z_t and their bounds B_t to construct an evidence process that provides an anytime-valid upper confidence bound U_t^J on $|J|$. This bound remains valid across all audit rounds with probability at least $1-\delta$, enabling reliable recall certification under adaptive stopping.

An Anytime-Valid Upper Bound on $|J|$. After audit t, our goal is to obtain an upper bound on the total match count $|J|$ with failure probability at most δ. Although many such bounds are possible, their tightness directly affects when ADAPT can stop verification. For example, by Theorem 3, B_t provides a conservative upper bound on $|J|$. However, a looser upper bound leads to a lower recall estimate, potentially delaying certified stopping and increasing LLM verification cost. We therefore aim to construct the tightest possible upper confidence bound U_t^J after each audit t:

$$\text{minimize } U_t^J, \quad \text{s.t. } \Pr\big(\forall t:\ |J| \le U_t^J\big) \ge 1-\delta. \tag{13}$$

To construct U_t^J, we develop a two-stage method. First, we introduce an *upper-bound testing algorithm* based on an e-process [13]. For each candidate integer u, the algorithm tests whether the accumulated audit evidence supports u as an upper bound on $|J|$, with the failure probability controlled at δ, as required by Equation 13. Second, we employ *binary search* to efficiently identify the smallest candidate upper bound that passes the test, yielding U_t^J.

(1) Upper Bounds Testing Algorithm. At audit t, for each candidate integer $u < \min\{B_i\}_{i=1}^t$, ADAPT uses an e-process to accumulate evidence across audit rounds and determine whether u can be certified as an upper bound on $|J|$. Based on the two properties of Z_t in Equation 11, we construct a component e-process $E_t^{(\lambda)}(u)$ with a fixed fraction $\lambda \in (0,1)$, following the betting confidence sequence framework [13]:

$$E_t^{(\lambda)}(u) = \prod_{i=1}^{t}\left(1+\lambda\frac{u-Z_i}{B_i-u}\right), \qquad E_0^{(\lambda)}(u) = 1. \tag{14}$$

To eliminate reliance on the hyper-parameter λ, ADAPT selects a finite set of fractions $\Lambda \subset (0,1)$, with each $\lambda \in \Lambda$ defining

a component process $E_t^{(\lambda)}(u)$ and $\pi_\lambda \geq 0$ denoting its weight in the *fixed-weight mixture* [13]:

$$E_t(u) = \sum_{\lambda \in \Lambda} \pi_\lambda E_t^{(\lambda)}(u), \qquad \sum_{\lambda \in \Lambda} \pi_\lambda = 1. \tag{15}$$

ADAPT uses $\Lambda = \{0.05, 0.10, 0.25, 0.50, 0.75\}$ and assigns equal weights $\pi_\lambda = 1/|\Lambda| = 1/5$ to all components. These fractions and weights remain fixed across all audits and rounds.

For each candidate integer u, we consider the *null hypothesis* $H_0(u) : |J| \geq u$ and construct the corresponding e-process $E_t(u)$ to accumulate evidence. If $H_0(u)$ is true, Ville's inequality [17] gives:

$$\Pr_{H_0(u):|J|\geq u}(\exists t : \ E_t(u) \geq 1/\delta) \leq \delta. \tag{16}$$

According to Equation 16, if $E_t(u)$ reaches $1/\delta$ at any audit t, we can reject $H_0(u)$ at significance level δ in favor of its *alternative hypothesis* $H_1(u) : |J| \leq u - 1$, and admit $u - 1$ as a certified candidate upper bound on $|J|$. If $E_t(u)$ has not reached $1/\delta$, we cannot reject $H_0(u)$, and $u - 1$ remains uncertified by this test.

(2) Binary Search for the Tightest Upper Bound U_t^J. To construct U_t^J, we seek the smallest integer u^* satisfying $E_t(u^*) \geq 1/\delta$, which yields a certified upper bound of $u^* - 1$ at audit t. Since $E_t(u)$ is nondecreasing in u by Equation 14, we can efficiently find u through *binary search* over integer candidates using the accumulated audit history $\mathcal{H}_t$. Then we obtain the tightest upper bound $U_t^J = \min\{U_{t-1}^J, u^* - 1\}$ at audit t. If no candidate integer u passes the test, we retain the previous upper bound $U_t^J = U_{t-1}^J$. Initially, $U_0^J = |L||R|$ and we progressively tighten U_t^J as audit evidence accumulates.

Certified Stopping. To decide whether the updated result after audit t meets the recall target $\tau_\mathcal{R}$, ADAPT computes a recall lower bound $\underline{\mathcal{R}}_t$ by dividing the verified match count $|\widehat{J}_t|$ by the upper bound U_t^J after every audit t. The system stops once it reaches $\tau_\mathcal{R}$:

$$\underline{\mathcal{R}}_t = \frac{|\widehat{J}_t|}{U_t^J}, \qquad \textsc{Stop}_t = \mathbf{1}[\underline{\mathcal{R}}_t \geq \tau_\mathcal{R}]. \tag{17}$$

When $U_t^J = 0$, we define $\underline{\mathcal{R}}_t = 1$ by convention.

Stratified Auditing for Reducing LLM Cost. The certification procedure above leverages verification feedback to estimate match probabilities f_t and determine audit sampling probabilities q_t. Although uniform sampling gives every unverified pair a chance of being audited, it ignores differences in match likelihood across pairs. To address this issue, we introduce *stratified auditing*, which exploits these differences to allocate audit samples more efficiently and reduce LLM verification cost.

(1) Stratified Auditing for $f_t(x)$. In Equation 10, assigning the same match probability to all pairs $x \in \mathcal{U}_t$ may lead to inaccurate estimates, as different pairs have different probabilities of being matches. To this end, based on the retrieval results from the current round, ADAPT partitions the unverified pair space $\mathcal{U}_t$ into three disjoint regions: (1) The *unverified queue* $\mathcal{C}_t$ contains unverified pairs retained in the candidate queues. (2) The *retrieved tail* $\mathcal{T}_t$ contains pairs retrieved through embedding-based search but not retained in any candidate queue. (3) The *unseen region* $\mathcal{D}_t$ contains all remaining unverified pairs. The first two regions are explicitly materialized during candidate generation, while the unseen region is represented implicitly, with its size computed as $|\mathcal{U}_t| - |\mathcal{C}_t| - |\mathcal{T}_t|$. This partitioning avoids computing similarity scores for every pair in $L \times R$, enabling scalable auditing.

For each region $\mathcal{S} \in \{\mathcal{C}_t, \mathcal{T}_t, \mathcal{D}_t\}$, ADAPT estimates its match rate $f_t(\mathcal{S}) \in [0, 1]$ by applying Beta smoothing [18] to historical audit labels. We then set $f_t(x) = f_t(\mathcal{S})$ for all pairs $x \in \mathcal{S}$. This allows the predicted number of remaining matches in Equation 10 to be computed efficiently using only three region-level summaries:

$$\sum_{x \in \mathcal{U}_t} f_t(x) = \sum_{\mathcal{S} \in \{\mathcal{C}_t, \mathcal{T}_t, \mathcal{D}_t\}} f_t(\mathcal{S})|\mathcal{S}|. \tag{18}$$

Thus, evaluating the estimator does not require enumerating the entire unverified space.

(2) Stratified Auditing for $q_t(x)$. In Equation 10, a straightforward approach is uniform sampling, where $q_t(x) = 1/|\mathcal{U}_t|$. However, different regions have different match rates and prediction errors. Allocating more audit samples to regions with higher prediction uncertainty can improve estimation efficiency and reduce LLM cost. To achieve this, we combine *uniform sampling* with *active allocation* [19]. *Uniform sampling* ensures that every unverified pair has a nonzero probability of being selected, while *active allocation* follows a Neyman-style strategy [20], assigning higher sampling probabilities to pairs in regions with greater prediction uncertainty:

$$q_t(x) = \frac{\rho}{|\mathcal{U}_t|} + (1 - \rho)\frac{\sqrt{v_t(x)}}{\sum_{z \in \mathcal{U}_t} \sqrt{v_t(z)}}, \quad \rho \in (0, 1]. \tag{19}$$

Here, $v_t(x)$ denotes the mean squared prediction error of the match-rate estimate f_t for the region containing x, and ρ controls the balance between uniform sampling and uncertainty-driven allocation.

Algorithm 1 summarizes the certified stopping procedure at audit round t. If no unverified pairs remain, the verified match count equals the exact total, and the algorithm stops without further auditing (Lines 1–3). Otherwise, it partitions the unverified pair space and prepares the match predictions f_t, sampling probabilities q_t, and estimator bound B_t (Lines 4–5). It then samples a pair according to q_t, obtains its LLM label, and computes the total match count estimate Z_t (Lines 6–7). The algorithm records the audit in the history and adds the sampled pair to the join result if it is verified as a match (Lines 8–9). Next, it uses binary search over the accumulated evidence to tighten the upper bound U_t^J (Line 10). Finally, it computes the recall lower bound, checks whether it reaches $\tau_\mathcal{R}$, and returns the stopping decision along with the updated join result, upper bound and history (Lines 11–13).

Theorem 4 (Anytime-Valid Certified Stopping)**.** *Algorithm 1 provides an anytime-valid stopping decision: the probability of stopping with recall below $\tau_\mathcal{R}$ is at most δ, satisfying Equation 7 even under adaptive retrieval and feature refinement.*

Algorithm 1: Certified Stopping Algorithm at Audit t

Input: Execution history $\mathcal{H}_{t^-}$ before audit t, previous bound U^J_{t-1}, current result $\widehat{J}_{t^-}$, unverified space $\mathcal{U}_{t^-}$, recall target $\tau_{\mathcal{R}}$, failure probability δ, LLM M_p

Output: Stopping decision $\textsc{Stop}_t$, updated result $\widehat{J}_t$, updated bound U^J_t, updated history $\mathcal{H}_t$ after audit t

if $\mathcal{U}_{t^-} = \emptyset$ **then**
  $\widehat{J}_t \leftarrow \widehat{J}_{t^-}$; $U^J_t \leftarrow |\widehat{J}_t|$; $\mathcal{H}_t \leftarrow \mathcal{H}_{t^-}$
  return TRUE, $\widehat{J}_t, U^J_t, \mathcal{H}_t$
$(\mathcal{C}_t, \mathcal{T}_t, \mathcal{D}_t) \leftarrow \texttt{Stratify}(\mathcal{U}_{t^-}, \mathcal{H}_{t^-})$
$(f_t, q_t, B_t) \leftarrow \texttt{Prepare}(\mathcal{C}_t, \mathcal{T}_t, \mathcal{D}_t, \mathcal{H}_{t^-})$ // Eq. (12)(18)(19)
$I_t \leftarrow \texttt{Sample}(\mathcal{U}_{t^-}, q_t)$; $y_t \leftarrow M_p(I_t)$ // Eq. (19)
$Z_t \leftarrow \texttt{EstimateJ}(\mathcal{H}_{t^-}, \mathcal{U}_{t^-}, f_t, q_t, y_t, I_t)$ // Eq. (10)
$\mathcal{H}_t \leftarrow \mathcal{H}_{t^-}.\texttt{Append}(I_t, y_t, Z_t, B_t)$
$\widehat{J}_t \leftarrow \widehat{J}_{t^-} \cup \{I_t : y_t = 1\}$
$U^J_t \leftarrow \texttt{BinarySearch}(\mathcal{H}_t, |\widehat{J}_t|, U^J_{t-1}, \delta)$ // Eq. (14)–(16)
$\underline{\mathcal{R}}_t \leftarrow |\widehat{J}_t| / U^J_t$
$\textsc{Stop}_t \leftarrow \mathbf{1}[\underline{\mathcal{R}}_t \geq \tau_{\mathcal{R}}]$ // Eq. (17)
return $\textsc{Stop}_t, \widehat{J}_t, U^J_t, \mathcal{H}_t$

Appendix D details the proof.

In each round, ADAPT runs at most T audits, which incurs the same cost as discovery. If the stopping condition cannot be met, it moves on to next step. All audit labels are reused as supervision for feature embedding refinement.

C. Heuristic Mode: Low-Cost Stopping Decision

Certified mode requires additional LLM calls to audit unverified pairs, and this cost can remain substantial even after most matches have been discovered. To reduce this overhead, heuristic mode estimates recall using only the existing *verified candidate stream*, without providing a formal quality guarantee. The key observation is that the rate of match discovery typically decreases as more candidate pairs are verified. As shown in Figure 3, the cumulative number of verified matches initially grows rapidly but gradually slows, suggesting a saturation trend. Motivated by this observation, *heuristic mode* performs decides when to stop through three steps: (i) Total Match Count Prediction by Fitting a Saturation Curve. (ii) Uncertainty Estimation via Residual Bootstrap. (iii) Recall Calculation and Stopping Decision.

Total Match Count Prediction by Fitting a Saturation Curve. To estimate the total match count $|J|$, ADAPT records the pairs verified by SW-UCB and their LLM labels in verification order, forming the *verified candidate stream*. Let x denote a pair's position in this stream and $C(x)$ the cumulative number of matches among the first x pairs. As verification progresses, the rate of match discovery typically decreases. Motivated by this trend, we fit a saturating exponential curve to the observed cumulative match counts after each round:

$$\widehat{C}(x) = \widehat{A}(1 - e^{-\beta x}), \tag{20}$$

where $\widehat{A}$ is the estimated saturation level, representing the predicted total number of matches and serving as an estimate of $|J|$. However, a single fitted estimate may be sensitive to fluctuations in the observed data and underestimate $|J|$,

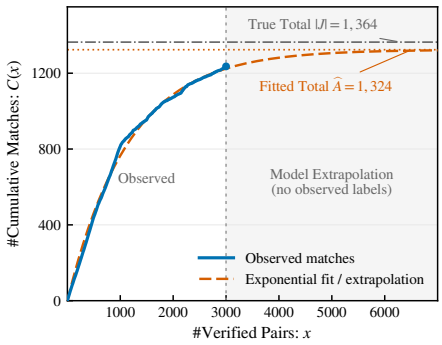


Fig. 3. A saturating exponential curve fitted to observed cumulative matches predicts future matches beyond the vertical cutoff. Horizontal lines indicate the estimated total $\widehat{A}$ and true total $|J|$.

potentially causing premature stopping. To account for this uncertainty, we use *residual bootstrap* to generate multiple perturbed trajectories and refit the curve to each.

Uncertainty Estimation via Residual Bootstrap. To obtain a more robust and conservative estimate of $|J|$, ADAPT uses *residual bootstrap* [21] to fit multiple saturation curves and takes the upper quantile of their estimated match counts. Specifically, we first compute the residuals $C(x) - \widehat{C}(x)$ at all observed points, forming a residual set $\mathcal{S}$. For each bootstrap iteration i, we sample residuals r_x with replacement from $\mathcal{S}$ and add them to the fitted values $\widehat{C}(x)$ to generate a perturbed trajectory. We then refit Equation 20 to obtain a new saturation estimate $\widehat{A}_i$. Repeating this process B times, we take the empirical $1-\delta$ quantile of the resulting estimates:

$$\widehat{A}_\delta = \text{Quantile}_{1-\delta}\big(\{\widehat{A}_i\}_{i=1}^{B}\big). \tag{21}$$

We use $B = 200$ bootstrap iterations in our experiments.

Recall Calculation and Stopping Decision. At the end of each round, ADAPT uses $\widehat{A}_\delta$ to calculate recall and check whether it reaches $\tau_{\mathcal{R}}$:

$$\widehat{\mathcal{R}}_{\text{H}} = \frac{|\widehat{J}|}{\widehat{A}_\delta}, \qquad \textsc{TargetMet} = \mathbf{1}[\widehat{\mathcal{R}}_{\text{H}} \geq \tau_{\mathcal{R}}]. \tag{22}$$

Here TARGETMET indicates whether the current-round recall criterion is satisfied. To determine whether to terminate, ADAPT additionally requires the absolute change in $\widehat{A}_\delta$ between the current and last rounds to be at most ϵ. Execution stops only when both the current-round recall criterion and this stability condition hold.

V. Cross-Channel Embedding Refinement

To improve candidate retrieval in subsequent rounds, ADAPT refines driving-side feature embeddings using accumulated LLM verification feedback. However, effective refinement within the progressive execution framework faces three challenges. First, the limited verification budget provides only a small amount of training data. Second, refining embeddings can be computationally expensive, requiring a lightweight model to minimize training overhead. Third, candidate retrieval favors pairs with high embedding similarity, introducing selection bias that increases the risk of overfitting. To address these challenges, we develop three complementary techniques:

(i) Training Data Augmentation. To mitigate limited and biased feedback, we leverage multi-channel retrieval to share verified pairs across channels. This allows each channel to learn from pairs discovered by other channels, particularly matching pairs it failed to retrieve. These missed matches provide corrective supervision for improving its feature embeddings. (ii) Lightweight Model Architecture. To reduce refinement cost, we design a lightweight residual MLP that learns additive adjustments to the driving-side embeddings produced by a pretrained encoder. By preserving the original embeddings when the learned adjustments are zero, this architecture retains the pretrained encoder's semantic knowledge while enabling efficient, targeted refinement. (iii) Regularized Training. To reduce overfitting to limited and biased feedback, we introduce regularization techniques at three stages: before, during, and after training. These techniques improve the robustness of embedding refinement while maintaining low computational overhead.

Training Data Augmentation. ADAPT maintains a separate training dataset for each retrieval channel. Since refinement operates on feature embeddings, each training example consists of a driving-side feature embedding, a driven-side record embedding, and a verified label. However, a driving-side record l_i may have multiple feature embeddings within a channel. Thus, a verified record pair (l_i, r_j) discovered by another channel does not directly identify which feature embedding should be used for training the target channel. To enable cross-channel feedback sharing, we map each verified record pair to a representative feature pair in each target channel. Specifically, for each verified record pair (l_i, r_j) and channel c, let X_i^c denote the set of original feature embeddings of l_i in channel c, and let y_j denote the embedding of r_j. We select the feature embedding most similar to y_j: $u_{ij}^c = \arg\max_{x \in X_i^c} \cos(x, y_j)$. The resulting feature pair (u_{ij}^c, y_j) inherits the verified label of (l_i, r_j): 1 for a match and 0 for a non-match. To avoid redundant computation, we cache and reuse these mappings across rounds, since the original embeddings remain fixed throughout execution. New mappings are computed only for newly verified record pairs.

Lightweight Model Architecture. To enable low-cost embedding refinement, ADAPT trains a lightweight residual MLP [22] for each channel c. The model learns additive adjustments to the driving-side feature embeddings, while keeping all driven-side embeddings fixed. Given an original feature embedding x, the refined embedding is computed as

$$g_{\theta_c}(x) = \text{normalize}\left(x + h_{\theta_c}(x)\right), \tag{23}$$

where $h_{\theta_c} : \mathbb{R}^d \to \mathbb{R}^d$ is a three-layer MLP that learns the embedding adjustment. We initialize the weights and biases of its final layer to zero, ensuring that $g_{\theta_c}(x)$ initially preserves the original embedding (assuming x is normalized). This initialization retains the semantic information learned by the pretrained encoder while allowing the model to progressively refine embeddings using verification feedback.

Regularized Training Method. We train the residual MLP using contrastive learning with the standard InfoNCE loss [14]. To construct contrastive learning samples, for each positive pair (x_a, y^+), where x_a is the anchor and y^+ is the positive target, we select a negative pair (x_a, y^-), where y^- is the embedding of a driven-side record that does not match the driving-side record l_i associated with embedding x_a. When insufficient verified negatives are available, we supplement them with *hard negatives* [23]: driven-side embeddings that are highly similar to the anchor but have not been verified as matches for l_i.

However, the limited and biased verification feedback makes the model prone to overfitting when trained directly with InfoNCE. To improve robustness, we introduce regularization techniques at three stages: before, during, and after training.

(1) Before Training: Fresh Initialization. For each channel, we initialize a new residual MLP and train it using the original embeddings. We avoid reusing the model or refined embeddings from previous rounds, as doing so may accumulate errors and reinforce biases in the verification feedback. If too few positive examples are available, we skip training and retain the original embeddings for retrieval in the next round.

(2) During Training: Validation-Guided Model Selection. Since a lower training loss does not necessarily indicate better retrieval quality, we evaluate the model after each epoch using filtered Mean Reciprocal Rank (MRR) [24] on a validation set. We apply early stopping when validation MRR fails to improve for a predefined number of epochs and retain the checkpoint with the highest validation MRR. To avoid degrading retrieval performance, we further compare the selected model against the original embeddings on the same validation set. We accept the refined model only if it achieves a higher MRR; otherwise, we retain the original embeddings for the next round.

(3) After Training: Displacement Control. Even when the refined model improves validation performance, large embedding adjustments may distort the semantic information captured by the pretrained encoder. To limit this risk, we constrain the magnitude of each embedding adjustment using a predefined threshold $\eta > 0$. Specifically, given an original embedding x and its refined embedding $g_{\theta_c}(x)$, we compute

$$\begin{aligned} a_x &= \min\left\{1, \frac{\eta}{\|g_{\theta_c}(x) - x\|_2}\right\}, \\ x' &= \text{normalize}(x + a_x(g_{\theta_c}(x) - x)). \end{aligned} \tag{24}$$

Here, x' is the final refined embedding, and we set $a_x = 1$ when $g_{\theta_c}(x) = x$. This constraint limits the adjustment from the original embedding, reducing the risk of overfitting while preserving its underlying semantic information.

Finally, ADAPT uses the refined driving-side embeddings to query the unchanged driven-side index and construct candidate queues for the next round, following Step 2 in Section III.

VI. EXPERIMENTS

A. Experimental Setup

1) Experimental Environment: We use the open-source Qwen3.5-397B-A17B model [25], self-hosted on NVIDIA

TABLE I
DATASETS. $\overline{w}$ IS THE AVG NUMBER OF WORDS PER RECORD.

Dataset	L		R		#Matches	Selectivity ($\times 10^{-3}$)
	$\|L\|$	$\overline{w}$	$\|R\|$	$\overline{w}$		
MovieActor	500	158.25	1,109	85.02	1,364	2.46
MovieDirector	500	116.98	1,158	102.17	1,166	2.01
ParentChild	500	157.56	469	195.84	566	2.41
Spouse	500	161.48	501	192.73	518	2.07
BioDEX	500	2,055.51	1,114	2.19	3,278	5.89
Wikipedia	500	868.73	1,053	4.34	1,345	2.55

H200 GPU hardware, as the LLM, and e5-base-v2 [26] as the embedding model. ADAPT is implemented in Python, using PyTorch for feature refinement and Faiss [27] for vector retrieval.

2) Datasets: Table I summarizes the six datasets. We construct (i) *MovieActor*, (ii) *MovieDirector*, (iii) *ParentChild*, and (iv) *Spouse* from T-REx [28], which aligns natural-language text with knowledge-base triples. The four datasets differ in both predicates and entity domains: *MovieActor* and *MovieDirector* match people to creative works through acting and directing relations, respectively, whereas *ParentChild* and *Spouse* match people to people through familial and marital relations. (v) *BioDEX* [29] joins biomedical articles with medical reaction terms by determining whether the patient responses described in an article match a given term. (vi) *Wikipedia* [30] joins articles with category labels by determining whether each article belongs to a given category.

3) Baseline Methods: We compare ADAPT with the following semantic-join methods.

- **LOTUS** [1] uses embedding-based cascades to reduce LLM comparisons, calibrating decision thresholds on sampled pairs to meet user-specified quality targets.
- **DocETL** [4] uses embedding-based and code-based blocking to prune candidate pairs before LLM evaluation.
- **FDJ** [11] approximates the join predicate with logical expressions over extracted features and uses LLM to refine the resulting candidate set. It provides a statistical guarantee that the final result meets a user-specified recall target relative to LLM labels, with a bounded failure probability.

We evaluate two variants of ADAPT:

- **ADAPT-C**: *Certified Mode* performs randomized auditing to support anytime-valid stopping relative to LLM labels;
- **ADAPT-H**: *Heuristic Mode* estimates recall from previously verified pairs without additional audit calls or guarantee.

4) Evaluation Metrics: We report two groups of metrics: (i) *Effectiveness.* Precision, recall, and F1 against the dataset ground truth to account for errors in the LLM's judgments. (ii) *Efficiency.* Total LLM calls, total input and output tokens, and LLM dollar cost.

B. End-to-End Experiments

Table II compares the end-to-end quality and LLM cost of ADAPT-C and ADAPT-H against LOTUS, DocETL, and FDJ across six datasets. We have three observations. First, ADAPT-H consistently incurs **the lowest cost** across all datasets, followed by ADAPT-C, and simultaneously **outperform every baseline in F1**, which demonstrates the superiority of our framework. Compared with the cheapest baseline on each workload, their cost reductions are 3.7-12.7× and 2.1-8.8×, respectively. The pattern is because of ADAPT's design: multiple feature channels and iterative feedback concentrate verification on promising candidates, while quality-aware stopping limits further calls. The savings therefore accompany a better efficiency-effectiveness balance.

Second, ADAPT attains the best or second-best ground-truth precision and recall on most datasets in Table II; only a few results fall outside. These exceptions should be interpreted in light of its quality definition and stopping objective. Because every returned pair is verified by the LLM, precision is 1 relative to LLM labels, whereas ground-truth precision depends on agreement between LLM labels and dataset annotations. BioDEX requires specialized medical knowledge, and Wikipedia category assignments can be subjective, making LLM errors more likely and reducing ground-truth precision. The other four datasets cover fact-based movie and social relational tasks, where LLMs tend to make few errors and achieve high precision. Recall is target-driven rather than maximized: ADAPT stops once its configured LLM-relative F1 target is met. Lower recall than the baseline on certain datasets does not imply a failure to satisfy the intended quality target. A more stringent target can trade extra cost for improved recall.

Third, our two modes yield similar F1, and ADAPT-H is consistently cheaper. ADAPT-C performs randomized auditing to provide certified stopping, while ADAPT-H estimates quality from existing verification feedback. Their cost gap varies across workloads because both audit requirements and stopping times affect total verification effort. Certification does not ensure higher ground-truth recall than ADAPT-H, as the Wikipedia results show: the guarantee concerns fixed LLM labels, and audit feedback can alter subsequent candidate selection. ADAPT-C therefore trades additional cost for a formal guarantee, rather than a uniform increase in F1.

C. Anytime-Valid Quality Guarantee

In this section, we evaluate the reliability and efficiency of ADAPT's anytime-valid quality guarantee and compare it with FDJ, which also provides quality guarantees. Because the guarantee is defined relative to fixed LLM labels, in this section, we use the ground-truth labels as a deterministic surrogate for fixed LLM labels. This substitution avoids exhaustive LLM labeling, while the resulting quality metrics remain measured against the dataset ground truth.

Failure-Probability Calibration. We fix target recall at 0.90 and vary the nominal failure level δ from 0.02 to 0.20. For ADAPT-C, we replay 2,000 audit trajectories starting one positive short of the target after a frozen discovery ordering; certification before reaching the target counts as a failure. For FDJ, we run 2,000 calibration trials for each of three fixed feature decomposition and measure recall violations. Figure 4 shows that both methods' empirical failure rates, including their 95% confidence-interval upper bounds, remain below the nominal levels. We find ADAPT-C is conservative across the

TABLE II

END-TO-END COMPARISON ON SEMANTIC-JOIN WORKLOADS. C/H: CERTIFIED/HEURISTIC MODES. BOLD/UNDERLINED: BEST/SECOND-BEST. COST: $2/$6 PER M INPUT/OUTPUT TOKENS; PARENTHESES: ADAPT-H COST REDUCTION FACTORS VS. THE CHEAPEST BASELINE.

(a) Effectiveness.

Dataset	Precision					Recall					F1				
	DocETL	LOTUS	FDJ	ADAPT-C	ADAPT-H	DocETL	LOTUS	FDJ	ADAPT-C	ADAPT-H	DocETL	LOTUS	FDJ	ADAPT-C	ADAPT-H
MovieActor	0.904	0.771	0.800	0.917	**0.925**	0.831	0.840	0.927	**0.961**	0.954	0.866	0.804	0.859	**0.939**	**0.939**
MovieDirector	0.999	**1.000**	0.996	**1.000**	**1.000**	0.820	0.897	**0.978**	0.977	**0.978**	0.901	0.946	0.987	0.988	**0.989**
ParentChild	0.968	0.952	0.949	**0.974**	**0.974**	0.686	0.777	0.717	**0.806**	0.795	0.802	0.856	0.817	**0.882**	0.875
Spouse	0.989	0.652	0.926	**0.995**	**0.995**	0.529	0.315	**0.851**	0.849	0.849	0.689	0.424	0.887	**0.917**	**0.917**
BioDEX	0.086	**0.148**	0.086	0.115	0.119	**0.652**	0.277	0.218	0.643	0.638	0.151	0.193	0.124	0.195	**0.201**
Wikipedia	0.630	0.271	**0.743**	0.738	0.689	**0.874**	0.842	0.704	0.756	0.799	0.732	0.411	0.723	**0.747**	0.740

(b) Efficiency.

Dataset	#Calls					#Tokens					Cost ($)				
	DocETL	LOTUS	FDJ	ADAPT-C	ADAPT-H	DocETL	LOTUS	FDJ	ADAPT-C	ADAPT-H	DocETL	LOTUS	FDJ	ADAPT-C	ADAPT-H
MovieActor	70,681	98,075	76,695	13,313	**4,441**	39.2M	44.6M	35.8M	8.25M	**2.78M**	78.89	90.73	72.48	16.63	**5.69** (12.7×)
MovieDirector	32,180	67,442	69,710	5,698	**4,637**	16.1M	31.4M	27.9M	3.43M	**2.86M**	32.42	63.96	56.62	6.93	**5.80** (5.6×)
ParentChild	12,738	25,705	128,256	5,346	**2,005**	8.70M	12.9M	76.9M	4.10M	**1.56M**	17.50	26.15	155	8.26	**3.17** (5.5×)
Spouse	13,365	25,760	107,250	5,275	**2,505**	9.01M	26.6M	67.1M	4.06M	**1.97M**	18.13	53.71	135	8.18	**4.00** (4.5×)
BioDEX	432,459	87,629	321,943	58,005	**33,005**	1.54B	446M	1.17B	210M	**121M**	3,087	893	2,344	421	**243** (3.7×)
Wikipedia	115,402	118,038	79,898	8,579	**8,005**	179M	169M	124M	14.1M	**13.4M**	359	339	249	28.44	**27.03** (9.2×)

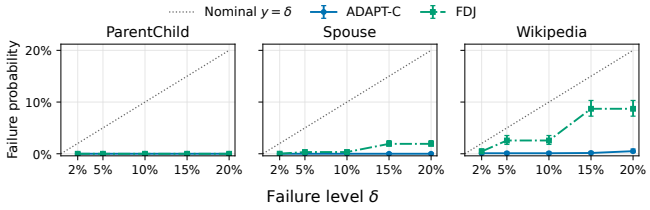


Fig. 4. Failure-probability calibration at target recall 90%. Markers indicate empirical failure rates, and error bars show 95% confidence intervals.

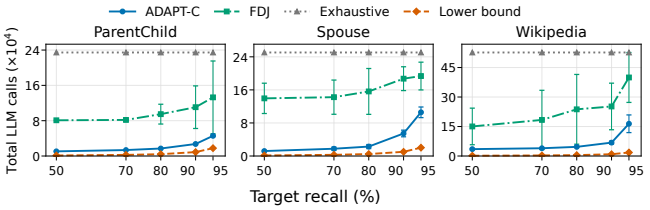


Fig. 5. Total LLM calls across recall targets ($\delta = 0.1$). Markers: mean LLM calls; error bars: show ±1 standard deviation across repeated runs.

tested settings, with no observed failures on ParentChild or Spouse and at most 0.5% on Wikipedia. These results support its calibration under repeated audit checks. We find ADAPT-C has lower observed failure rates than FDJ on Spouse and Wikipedia, and provides a more reliable guarantee.

Total LLM Calls. We next fix $\delta = 0.10$ and vary target recall from 0.50 to 0.95. The count includes feature preparation and all distinct pair verifications, including discovery and auditing for ADAPT-C, rather than certification alone. Figure 5 shows that ADAPT-C uses fewer calls than FDJ and exhaustive verification at every plotted target, while both ADAPT-C and FDJ meet their recall targets in all recorded runs. We find that FDJ's cost varies substantially across runs on all three datasets, because it uses random sampling to find the positive examples for feature decomposition; different samples yield different decompositions and, consequently, different verification costs. In contrast, we find that ADAPT-C achieves both lower and more stable cost, because its progressive feedback loop iteratively refines retrieval using accumulated verification outcomes rather than relying on a predefined plan.

D. Scalability

We vary the Wikipedia table sizes so that the total join space size $N = |L| \times |R|$ ranges from 10^5 to 10^7, and measure total LLM calls and ground-truth recall. Figure 6 shows that call counts grow much more slowly than the join space: a $100\times$ increase in N requires only about $14.5\times$ as many calls. Meanwhile, recall remains approximately 70%, indicating that the reduced growth in calls does not come with a substantial loss of recall over the tested sizes.

To evaluate scalability, we examine how the number of LLM calls grows with the join space size N under a fixed recall target. We fit two models, $c\sqrt{N}\log N$ and $cN\log N$, to the observed call counts and find that the former fits more closely ($R^2 = 0.99$ versus 0.74). The fitted trend suggests slow, sublinear growth in LLM calls, supporting the scalability of ADAPT over the tested range.

E. Ablation Studies

We evaluate the contributions of multi-channel features and cross-channel refinement on Wikipedia-Large and BioDEX-Large, each containing 500 driving records and 5,000 driven records. We use these larger datasets to increase task complexity and better assess the performance benefits of our method. All variants run for ten rounds with 2,500 pair verifications per round.

1) Effect of Multi-Channel Features: We compare the multi-channel feature with variants using only keywords, sentences, documents, or LLM-extracted features. Figure 7 shows that multi-channel features achieves the highest final recall on both workloads. It exceeds LLM-Ext, the strongest individual view, by 6.8 percentage points on Wikipedia-Large and 4.0 points on BioDEX-Large. On BioDEX-Large, the two curves are nearly identical initially, but the multi-view advantage emerges as execution progresses. Thus, the benefit

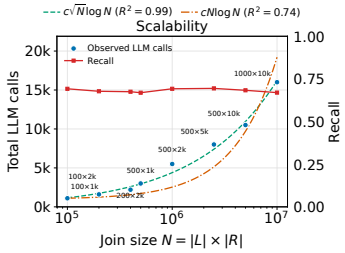


Fig. 6. Scalability(Wikipedia)

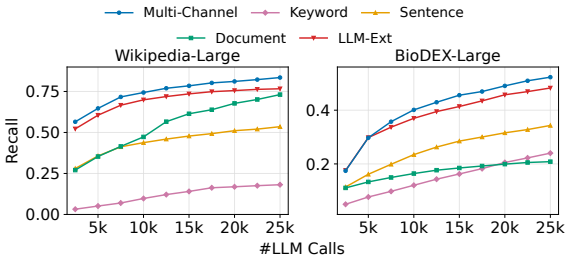


Fig. 7. Effect of Multi-Channel Features

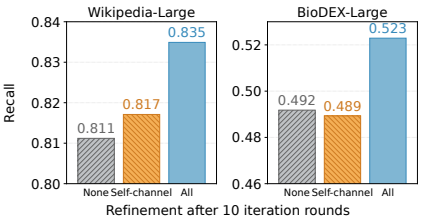


Fig. 8. Effect of Feature Refinement

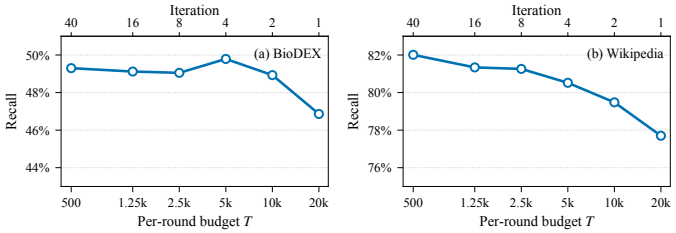


Fig. 9. Effect of Per-Round Budget T

extends beyond the initial candidate ranking, consistent with iterative use of complementary views and feedback. The other views also behave differently across workloads: document features approach LLM-Ext on Wikipedia-Large but perform poorly on BioDEX-Large, where sentence features are more effective. This suggests that whole-document similarity is less informative for some predicates, supporting multiple retrieval views rather than reliance on one granularity.

2) Effect of Feature Refinement: Keeping all four channels, we compare no refinement (None), refinement using only each channel's own feedback (Self-channel), and refinement using feedback from all channels (All). Figure 8 reports recall after ten rounds at the same verification budget. All-channel refinement performs best on both workloads, improving recall over no refinement by 2.4 and 3.1 percentage points, respectively. Self-channel refinement yields only a small gain on Wikipedia-Large and slightly reduces recall on BioDEX-Large, so updating embeddings alone does not consistently help. The advantage of shared feedback is consistent with the cross-channel design: matches discovered by one channel provide training examples that another may otherwise miss. The two ablations support both maintaining complementary retrieval channels and sharing their verification feedback during refinement.

F. Parameter Sensitivity

We fix the total LLM budget at 20,000 calls and vary the per-round budget T from 500 to 20,000, yielding 40 rounds to a single round. All configurations use the same seed. Figure 9 shows stable recall for $T \leq 5{,}000$ and lower recall at larger budgets. Increasing T reduces the number of rounds, leaving fewer opportunities to refine embeddings with cross-channel feedback and improve subsequent candidate retrieval. Single-round execution yields the lowest recall on both datasets: although feedback still guides channel selection, no subsequent retrieval benefits from refinement. The relatively stable recall at smaller budgets suggests diminishing returns from more frequent updates, so more rounds do not necessarily improve recall. Our default $T = 500$ enables repeated refinement and achieves recall close to the best observed result on both datasets.

VII. Related Work

Semantic Join Optimization. LOTUS [1] reduces LLM calls through embedding-based cascades with calibrated thresholds. PZ's Abacus optimizer [2], [3] selects between LLM-based nested-loop and embedding joins using sampled quality, cost, and latency estimates. DocETL [4] prunes candidate pairs through code-based and embedding-based blocking. FDJ [11] combines predicate-relevant feature comparisons into logical filters with statistical recall guarantees. However, these workflows do not continually reuse execution-time verification labels to improve subsequent candidate generation. Weaknesses in the configured proxies can therefore persist: restrictive rules may miss matches, while permissive rules retain many non-matches for costly LLM verification.

Statistical Quality Guarantees. For semantic filters, SUPG [31] calibrates proxy thresholds using sampled oracle labels to meet precision or recall targets. BARGAIN [32] improves cascade certification through adaptive sampling and sequential statistical tests. These guarantees concern threshold-defined outputs and do not directly certify the accumulated verified matches at arbitrary stopping points in a progressive join. For semantic joins, LOTUS [1] provides quality guarantees through calibrated embedding cascades, while FDJ [11] accounts for feature dependencies when constructing filters with recall guarantees. Their guarantees apply to execution under calibrated rules; feedback-driven changes to representations and candidate selection require additional analysis to maintain validity across rounds and data-dependent stopping.

VIII. Conclusion

We presented ADAPT, an adaptive feedback-aware framework for semantic joins. It combines multi-feature retrieval, cross-channel refinement, and adaptive verification-budget allocation, reusing LLM verification labels to improve candidate discovery. Its certified mode provides an anytime-valid quality guarantee relative to fixed LLM labels through randomized auditing, while its heuristic mode avoids additional audit calls. Experiments on six workloads show that both modes achieve higher F1 and lower LLM cost than all baselines: *Certified mode* reduces cost by 2.1–8.8×, while *heuristic mode* reduces cost by 3.7–12.7×.

ACKNOWLEDGMENTS

AI Assistance Statement. OpenAI Codex was used only to assist with code implementation. No AI assistance was used for method design, experiment execution, or manuscript writing. The authors take full responsibility for the code and the reported results.

REFERENCES

[1] L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia, "Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4171–4184, 2025.

[2] C. Liu, M. Russo, M. Cafarella, L. Cao, P. B. Chen, Z. Chen, M. Franklin, T. Kraska, S. Madden, R. Shahout, and G. Vitagliano, "Palimpzest: Optimizing AI-powered analytics with declarative query processing," in *Proceedings of the 15th Conference on Innovative Data Systems Research (CIDR)*, 2025.

[3] M. Russo, C. Liu, S. Sudhir, G. Vitagliano, M. J. Cafarella, T. Kraska, and S. Madden, "Abacus: A cost-based optimizer for semantic operator systems," *Proceedings of the VLDB Endowment*, vol. 19, no. 5, pp. 1060–1073, 2026. [Online]. Available: https://www.vldb.org/pvldb/vol19/p1060-russo.pdf

[4] S. Shankar, T. Chambers, T. Shah, A. G. Parameswaran, and E. Wu, "DocETL: Agentic query rewriting and evaluation for complex document processing," *Proc. VLDB Endow.*, vol. 18, no. 9, pp. 3035–3048, 2025.

[5] Z. Sun, J. Wang, X. Zhao, J. Wang, and G. Li, "Data agent: A holistic architecture for orchestrating data+ai ecosystems," *CoRR*, vol. abs/2507.01599, 2025.

[6] Z. Sun, X. Wang, and G. Li, "Agentic transaction: Towards acid-compliant agent systems," *arXiv preprint arXiv:2608.13900*, 2026.

[7] X. Zhou, Z. Sun, and G. Li, "Db-gpt: Large language model meets database," *Data Science and Engineering*, vol. 9, no. 1, pp. 102–111, 2024.

[8] S. Xu, J. Wang, and G. Li, "Bridging the gap: Cardinality estimation for semantic queries on unstructured data," *Proc. ACM Manag. Data*, vol. 4, no. 3, pp. 147:1–147:26, 2026. [Online]. Available: https://doi.org/10.1145/3802024

[9] J. Wang, G. Li, and J. Feng, "idatalake: An llm-powered analytics system on data lakes," *IEEE Data Eng. Bull.*, vol. 49, no. 1, pp. 57–69, 2025. [Online]. Available: http://sites.computer.org/debull/A25mar/p57.pdf

[10] J. Wang and G. Li, "AOP: automated and interactive LLM pipeline orchestration for answering complex queries," in *15th Conference on Innovative Data Systems Research, CIDR 2025, Amsterdam, The Netherlands, January 19-22, 2025*. www.cidrdb.org, 2025. [Online]. Available: https://vldb.org/cidrdb/2025/aop-automated-and-interactive-llm-pipeline-orchestration-for-answering-complex-queries.html

[11] S. Zeighami, S. Shankar, and A. G. Parameswaran, "Featurized-decomposition join: Low-cost semantic joins with guarantees," *Proceedings of the VLDB Endowment*, vol. 19, no. 11, pp. 3048–3061, 2026. [Online]. Available: https://doi.org/10.14778/3836663.3836672

[12] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in *Proceedings of the 22nd International Conference on Algorithmic Learning Theory*, 2011, pp. 174–188.

[13] I. Waudby-Smith and A. Ramdas, "Estimating means of bounded random variables by betting," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 86, no. 1, pp. 1–27, 2024.

[14] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[15] T. Zrnic and E. Candes, "Active statistical inference," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 62 993–63 010. [Online]. Available: https://proceedings.mlr.press/v235/zrnic24a.html

[16] A. Fisch, J. Maynez, R. A. Hofer, B. Dhingra, A. Globerson, and W. W. Cohen, "Stratified prediction-powered inference for effective hybrid evaluation of language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[17] J. Ville, *Étude critique de la notion de collectif*. Paris: Gauthier-Villars, 1939. [Online]. Available: https://www.numdam.org/item/THESE_1939__218__1_0/

[18] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, *Bayesian Data Analysis*, 3rd ed. Chapman & Hall/CRC, 2013. [Online]. Available: https://sites.stat.columbia.edu/gelman/book/

[19] P. Li, T. Zrnic, and E. Candes, "Robust sampling for active statistical inference," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[20] J. Neyman, "On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection," *Journal of the Royal Statistical Society*, vol. 97, no. 4, pp. 558–606, 1934.

[21] D. A. Freedman, "Bootstrapping regression models," *The Annals of Statistics*, vol. 9, no. 6, pp. 1218–1228, 1981.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

[23] J. Robinson, C.-Y. Chuang, S. Sra, and S. Jegelka, "Contrastive learning with hard negative samples," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=CR1XOQ0UTh-

[24] E. M. Voorhees, "The TREC-8 question answering track report," in *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*, ser. NIST Special Publication, vol. 500-246. National Institute of Standards and Technology, 1999. [Online]. Available: https://trec.nist.gov/pubs/trec8/papers/qa_report.pdf

[25] Qwen Team, "Qwen3.5: Towards native multimodal agents," Feb. 2026. [Online]. Available: https://qwen.ai/blog?id=qwen3.5

[26] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022. [Online]. Available: https://arxiv.org/abs/2212.03533

[27] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," *arXiv preprint arXiv:2401.08281*, 2024. [Online]. Available: https://arxiv.org/abs/2401.08281

[28] H. Elsahar, P. Vougiouklis, A. Remaci, C. Gravier, J. Hare, F. Laforest, and E. Simperl, "T-REx: A large scale alignment of natural language with knowledge base triples," in *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. Miyazaki, Japan: European Language Resources Association (ELRA), 2018. [Online]. Available: https://aclanthology.org/L18-1544/

[29] K. D'Oosterlinck, F. Remy, J. Deleu, T. Demeester, C. Develder, K. Zaporojets, A. Ghodsi, S. Ellershaw, J. Collins, and C. Potts, "BioDEX: Large-scale biomedical adverse drug event extraction for real-world pharmacovigilance," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, pp. 13 425–13 454. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.896/

[30] K. Bhatia, K. Dahiya, H. Jain, P. Kar, A. Mittal, Y. Prabhu, and M. Varma, "The extreme classification repository: Multi-label datasets and code," 2016. [Online]. Available: http://manikvarma.org/downloads/XC/XMLRepository.html

[31] D. Kang, E. Gan, P. Bailis, T. Hashimoto, and M. Zaharia, "Approximate selection with guarantees using proxies," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 1990–2003, 2020.

[32] S. Zeighami, S. Shankar, and A. G. Parameswaran, "Cut costs, not accuracy: LLM-powered data processing with guarantees," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 311:1–311:26, 2025.

APPENDIX A
PROOF OF THEOREM 1

We treat the LLM labels as fixed throughout execution. All probabilities and expectations are over the algorithm's internal random choices, conditional on these labels and its initial information. We use the notation from the main text: $N = |L||R|$, $|J| = \kappa N$, and S is the total number of pair-level LLM verifications before termination. The true selectivity κ describes the instance but is not provided to the algorithm. To distinguish the two sources of verification cost, let S_+ and S_- count the distinct verified positive and negative pairs, respectively. Repeated queries reveal no new label, so

$$S \geq S_+ + S_-. \tag{A.1}$$

The lower bound applies to algorithms that almost surely terminate, return only LLM-verified matches, and satisfy Definition 1 for every label configuration compatible with their initial information. We additionally require that every alternative instance $J \cup A$ used below, where $A \subseteq (L \times R) \setminus J$ and $|A| = H$, is compatible with that same information. Thus, the quality requirement applies to every alternative in the argument. Only the final-output guarantee is needed; the proof does not require anytime-valid guarantees at intermediate checks.

Since the output contains only verified matches, Equations 4 and 6 imply that, with probability at least $1 - \delta$, the output contains at least $\lceil \tau_{\mathcal{R}} \kappa N \rceil$ verified positive pairs. Therefore,

$$\mathbb{E}[S_+] \geq (1 - \delta)\lceil \tau_{\mathcal{R}} \kappa N \rceil. \tag{A.2}$$

For the negative-evidence requirement, recall the threshold $H = \lfloor (1 - \tau_{\mathcal{R}})\kappa N / \tau_{\mathcal{R}} \rfloor + 1$ from Theorem 1. It satisfies $|J|/(|J| + H) < \tau_{\mathcal{R}}$: even returning every match in J would fail the target if H additional matches were hidden. The assumption $\kappa < \tau_{\mathcal{R}}$ ensures $1 \leq H \leq (1 - \kappa)N$.

Fix an instance J and let Q_- be the random set of negative pairs queried before stopping, so $|Q_-| = S_-$. For any $A \subseteq (L \times R) \setminus J$ with $|A| = H$, compare executions on J and $J \cup A$ using the same internal random seed. If $Q_- \cap A = \varnothing$, all observed labels are identical, so the algorithm stops with the same output on both instances. This output contains at most $|J|$ pairs and fails the recall target on $J \cup A$. The guarantee on every compatible alternative therefore implies

$$\Pr_J(Q_- \cap A = \varnothing) \leq \delta \qquad \text{for every such } A. \tag{A.3}$$

The subscript J indicates that the execution is on the fixed instance J.

Averaging over all H-element sets A gives

$$\mathbb{E}_J\left[\frac{\binom{(1-\kappa)N - S_-}{H}}{\binom{(1-\kappa)N}{H}} \right] \leq \delta. \tag{A.4}$$

For $S_- = s$, the ratio is the fraction of hidden-match sets that avoid the s queried negatives. To apply Jensen's inequality, define the auxiliary function

$$\phi(s) = \left(1 - \frac{s}{(1-\kappa)N - H + 1}\right)_+^H, \tag{A.5}$$

where $(\cdot)_+ = \max\{\cdot, 0\}$. This function is convex on $[0, (1 - \kappa)N]$ and, for each integer s in this interval, satisfies

$$\phi(s) \leq \frac{\binom{(1-\kappa)N - s}{H}}{\binom{(1-\kappa)N}{H}}. \tag{A.6}$$

Indeed, when $s \leq (1 - \kappa)N - H$, the ratio is a product of H factors, each at least $1 - s/((1 - \kappa)N - H + 1)$; for larger integer s, both sides are zero. Jensen's inequality then yields

$$\phi(\mathbb{E}_J[S_-]) \leq \mathbb{E}_J[\phi(S_-)] \leq \delta. \tag{A.7}$$

Rearranging gives

$$\mathbb{E}_J[S_-] \geq \big((1 - \kappa)N - H + 1\big)\big(1 - \delta^{1/H}\big). \tag{A.8}$$

Combining Equations A.1, A.2, and A.8 proves Equation 8 in the main paper.

For completeness, if $\kappa \geq \tau_{\mathcal{R}}$, then $H > (1 - \kappa)N$, so the required alternative instances cannot be constructed. Even if every negative pair were relabeled as a match, returning all original matches would achieve recall $|J|/N = \kappa \geq \tau_{\mathcal{R}}$. The hidden-match argument therefore gives no additional negative-evidence requirement in this regime, while Equation A.2 still applies. This explains the theorem's restriction to $\kappa < \tau_{\mathcal{R}}$; it does not imply that discovery incurs no negative verifications outside this regime. □

APPENDIX B
PROOF OF THEOREM 2

We simplify the negative-evidence requirement in Equation A.8 under the assumptions of Theorem 1. Fix $\tau_{\mathcal{R}}, \delta \in (0, 1)$ and suppose that the average number of matches per record in one input table stays positive and constant as both tables grow. If that table is L, then $|J| = \Theta(|L|)$, $H = \Theta(|L|)$, and $\kappa = \Theta(1/|R|)$. Consequently, $\log(1/\delta)/H \to 0$, $H \to \infty$, and $H/((1 - \kappa)N) \to 0$. The same reasoning applies with L and R exchanged, or in any other regime satisfying these three conditions.

Since $\log(1/\delta) \ll H$,

$$1 - \delta^{1/H} = 1 - e^{-\log(1/\delta)/H} = \Theta\left(\frac{\log(1/\delta)}{H}\right). \tag{B.1}$$

Moreover, $H \ll (1 - \kappa)N$ and $H \gg 1$ imply

$$\begin{aligned} (1 - \kappa)N - H + 1 &= \Theta((1 - \kappa)N), \\ H &= \Theta\left(\frac{1 - \tau_{\mathcal{R}}}{\tau_{\mathcal{R}}}\kappa N\right). \end{aligned} \tag{B.2}$$

Substituting into Equation A.8 gives

$$\mathbb{E}_J[S_-] = \Omega\left(\frac{1 - \kappa}{\kappa}\frac{\tau_{\mathcal{R}}}{1 - \tau_{\mathcal{R}}}\log\frac{1}{\delta}\right), \tag{B.3}$$

which proves Equation 9. This bound captures the audit cost of obtaining negative evidence in addition to the minimum cost of verifying enough positive pairs. Such evidence helps rule out alternative instances with hidden matches that would cause the returned result to fall below the recall target. Negative labels obtained through either candidate verification or dedicated auditing contribute to S_-. □

APPENDIX C
PROOF OF THEOREM 3

Fix an audit t with $\mathcal{U}_{t^-} \neq \emptyset$ and condition on $\mathcal{H}_{t^-}$. This fixes $\widehat{J}_{t^-}$, $\mathcal{U}_{t^-}$, f_t, and q_t before drawing I_t. The history includes all discovery labels and refinement updates since the preceding audit, so the argument allows retrieval and sampling to adapt across rounds. Throughout, the LLM labels and hence J are fixed.

By the sampling rule, I_t is drawn with probabilities $q_t(x) > 0$ that sum to one over $\mathcal{U}_{t^-}$. Taking the conditional expectation of Equation 10 gives

$$\begin{aligned}\mathbb{E}[Z_t \mid \mathcal{H}_{t^-}] &= |\widehat{J}_{t^-}| + \sum_{x \in \mathcal{U}_{t^-}} f_t(x) \\ &\quad + \sum_{x \in \mathcal{U}_{t^-}} q_t(x) \frac{M_p(x) - f_t(x)}{q_t(x)} \\ &= |\widehat{J}_{t^-}| + \sum_{x \in \mathcal{U}_{t^-}} M_p(x) = |J|.\end{aligned} \tag{C.1}$$

The correction cancels the total prediction error in conditional expectation. Thus, unbiasedness does not require accurate predictions, a fixed partition, or random candidate discovery.

For the upper bound, $M_p(I_t) \leq 1$ and $q_t(I_t) > 0$ imply

$$\begin{aligned}\frac{M_p(I_t) - f_t(I_t)}{q_t(I_t)} &\leq \frac{1 - f_t(I_t)}{q_t(I_t)} \\ &\leq \max_{x \in \mathcal{U}_{t^-}} \frac{1 - f_t(x)}{q_t(x)}.\end{aligned} \tag{C.2}$$

Adding the known and predicted match counts gives $Z_t \leq B_t$, with B_t defined in Equation 12. Every term in this bound is determined by $\mathcal{H}_{t^-}$, and the finite unverified space and positive sampling probabilities make B_t finite.

The stratified implementation satisfies these sampling conditions without enumerating the unverified space. Predictions and pair-level sampling probabilities are constant within each nonempty region $\mathcal{S} \in \{\mathcal{C}_t, \mathcal{T}_t, \mathcal{D}_t\}$. We select $\mathcal{S}$ with probability $\sum_{x \in \mathcal{S}} q_t(x)$ and draw uniformly within it, giving the required pair-level probability $q_t(x)$. The first two regions are sampled from their materialized sets; within $\mathcal{D}_t$, uniform draws from $L \times R$ are repeated until an unverified pair outside the first two regions is obtained. The selected region remains fixed during these proposals, so the accepted pair is uniform within it. Each audited pair is removed before the next draw. The uniform component ensures $q_t(x) \geq \rho/|\mathcal{U}_{t^-}|$; residual estimates are bounded below by a positive floor before active allocation is normalized. Consequently, the sum and maximum defining B_t can be evaluated from the three region summaries, while the same proof applies to any predictable sampling distribution with full support. □

APPENDIX D
PROOF OF THEOREM 4

Fix an integer threshold u and suppose $H_0(u) : |J| \geq u$ holds. For any $\lambda \in \Lambda$ and audit with $u < B_t$, the multiplier in Equation 14 is at least $1 - \lambda > 0$. Theorem 3 gives

$$\begin{aligned}&\mathbb{E}\left[1 + \lambda \frac{u - Z_t}{B_t - u} \,\middle|\, \mathcal{H}_{t^-}\right] \\ &\qquad = 1 + \lambda \frac{u - |J|}{B_t - u} \leq 1.\end{aligned} \tag{D.1}$$

Each component $E_t^{(\lambda)}(u)$ is therefore a nonnegative test supermartingale starting at one. Its value is unchanged between audits; discovery and refinement only change the inputs to the next update, which are fixed before the next draw.

For the mixture in Equation 15, the fractions and nonnegative weights are fixed before auditing, with $\sum_{\lambda \in \Lambda} \pi_\lambda = 1$. Thus,

$$\begin{aligned}\mathbb{E}[E_t(u) \mid \mathcal{H}_{t^-}] &= \sum_{\lambda \in \Lambda} \pi_\lambda \mathbb{E}[E_t^{(\lambda)}(u) \mid \mathcal{H}_{t^-}] \\ &\leq \sum_{\lambda \in \Lambda} \pi_\lambda E_{t-1}^{(\lambda)}(u) \\ &= E_{t-1}(u).\end{aligned} \tag{D.2}$$

The mixture is also a nonnegative test supermartingale with $E_0(u) = 1$. Ville's inequality [13] therefore yields

$$\Pr_{H_0(u)}(\exists t : \ E_t(u) \geq 1/\delta) \leq \delta. \tag{D.3}$$

No division of the error budget among mixture components is needed.

The denominator in Equation 14 requires separate handling when $u \geq B_t$. If $u > B_t$, then $|J| = \mathbb{E}[Z_t \mid \mathcal{H}_{t^-}] \leq B_t < u$, so $H_0(u)$ is excluded directly. If $u = B_t$, $H_0(u)$ requires $|J| = B_t$; since $Z_t \leq B_t$, this implies $Z_t = B_t$ almost surely. Observing $Z_t < u$ therefore excludes the candidate, whereas observing $Z_t = u$ leaves its evidence unchanged. These rules avoid division by zero and have zero error probability when $H_0(u)$ is true.

For fixed audit records, each multiplier is nondecreasing in u on its domain because $Z_t \leq B_t$. Products of positive multipliers and their fixed-weight mixtures preserve this order, as do the exact boundary rules. Binary search can therefore find the smallest integer u^* in $\{|\widehat{J}_t|, \ldots, N\}$ certified by the evidence threshold or the boundary rules. Set $u^* = N + 1$ if no such integer exists. Retaining earlier bounds gives the update

$$U_t^J = \min\{U_{t-1}^J, u^* - 1\}, \qquad U_0^J = N. \tag{D.4}$$

If no candidate is certified, this update retains the previous bound. Between audits, the bound remains applicable to the same fixed $|J|$, while newly discovered matches update the recall numerator. If the unverified space is empty, $|J| = |\widehat{J}_t|$ is known exactly and no further audit is needed.

By monotonicity and retention of earlier bounds, an upper bound below $|J|$ implies that the threshold $u = |J|$ was certified at some audit:

$$\{\exists t : \ U_t^J < |J|\} \subseteq \{\exists t : \ E_t(|J|) \geq 1/\delta\}, \tag{D.5}$$

apart from the zero-probability boundary events. Applying Equation D.3 at this single true threshold establishes the

coverage constraint in Equation 13, without a union bound over candidate integers.

On the simultaneous coverage event, $U_t^J \geq |J|$, so Equation 17 gives a recall lower bound that never exceeds the true recall. Stopping only when this bound reaches $\tau_{\mathcal{R}}$ therefore has probability at most δ of returning a result below the target, as required by Equation 7. When J is empty, recall is defined as one, and the same guarantee holds. □